

LAB 9b

JAVASCRIPT 2: MORE DOM

What You Will Learn

- Additional DOM events
- How to implement common web user interface techniques

Approximate Time

The exercises in this lab should take approximately 45 minutes to complete.

Fundamentals of Web Development, 3rd Ed

Randy Connolly and Ricardo Hoar

Textbook by Pearson
<http://www.funwebdev.com>

Date Last Revised: August 25, 2025
Exercises 9b.5-9b.11 are new
Moved CSS filters test your knowledge to this lab and added snackbar requirement.

PREPARING DIRECTORIES

- 1 The starting `lab09b` folder has been provided for you (within the zip folder downloaded from Gumroad).

*Note: these labs use the convention of blue background text to indicate filenames or folder names and **bold red** for content to be typed in by the student.*

ADDITIONAL DOM EVENTS

Lab09a was a very long lab, so some of the less-used DOM events have been moved to this lab. These include keyboard events, frame events, drag events, and events specific to various browser APIs (that is, JavaScript objects provided by the browser that extend the capabilities of JavaScript).

EXERCISE 9b.1 — RESPONDING TO KEYBOARD EVENTS

- 1 Examine `lab09b-ex01.html` in your browser and text editor.

Notice the series of `<audio>` elements. Each of these elements contains a `data-key` attribute. The `data-*` attribute is a mechanism within HTML5 for adding custom information to an element. In this example, it will be used by our JavaScript for looking up the appropriate sound to play.

- 2 Open `lab09b-ex01.js` and add the following code and test by pressing the 'g' key.

```
document.addEventListener("DOMContentLoaded", function() {
    // when key is pressed play a sound if correct key
document.addEventListener('keydown', playSound);

    function playSound(e) {
        // initially let's just play the sound for data-key=g
        const audio = document.querySelector(
            `audio[data-key="g"]`);

        // since audio sound takes some time to play, we can reset
        // the current time to zero so that it is more responsive to
        // multiple fast key presses
        audio.currentTime = 0;

        // now play sound at beginning
        audio.play();
    }
});
```

There is not a lot of keyboard handling here: instead the focus is on selecting an `<audio>` element and playing the sound file specified in the element.

- 3 Modify the code as follows and test by pressing the keys a, s, d, f, g, h, j, k, l

```
function playSound(e) {
  // first determine the key that was pressed from event
  const keyPressed = e.key;

  // select the <audio> element for the pressed key
  const audio =
    document.querySelector(
      `audio[data-key="${keyPressed}"]`);

  // if key doesn't have <audio> element then exit
  if (!audio) return;
  ...
}
```

All nine sounds sound play when their associated key is pressed. Right now, there is no visual cue for the key being pressed. The next steps will make use of some already existing CSS classes to add some visual transition effects.

- 4 Modify the code as follows and test.

```
// if key doesn't have <audio> element then exit
if (!audio) return;

// select the <div> element corresponding to the pressed key
const div =
  document.querySelector(
    `div.key[data-key="${keyPressed}"]`);

// add glowing box class to it
div.classList.add('playing');
```

A glowing yellow box should now be showing up around the <div> on screen associated with the pressed key. The <div> also grows a bit in size. However, the effects never go away.

- 5 You can make the yellow box go away by responding to another event. Before doing that, open `lab09b-ex01.css` and look at the `key` and `playing` CSS classes. Notice the `transition` property in the `key` class? It specifies the length of time for all changes in visual state in the element assigned to this class. It is assigned to the <div> elements with the key and sound name (e.g., <div data-key="g" class="key">). The code in the previous step dynamically assigned the `.playing` class to the same <div> element, which means the `scale`, `border-color`, and `box-shadow` properties transitioned across `.07sec`. The ending of that transition is an event that can be responded to in JavaScript, which is covered in the next step.

6 Modify the code as follows and test:

```

window.addEventListener("DOMContentLoaded", function() {
  // when key is pressed play a sound if correct key
  document.addEventListener('keydown', playSound);

  // to remove the box around the div for the key, we will listen
  // for any transition-end event (in this case, used by the .key
  // class elements)
  const keys = document.querySelectorAll('.key');
  for (let key of keys) {
    key.addEventListener('transitionend', removeTransition)
  }

  function removeTransition(e) {
    // skip if not a transform transition
    if (e.propertyName !== 'transform') return;
    // right transition so remove the .playing class from .key
    this.classList.remove('playing');
  }

  ...

```

This exercise inspired from Wes Bos's JavaScript 30 sample project (<https://javascript30.com/>), and is used with permission.

It's possible the background image might not be visible (the unsplash source could change).

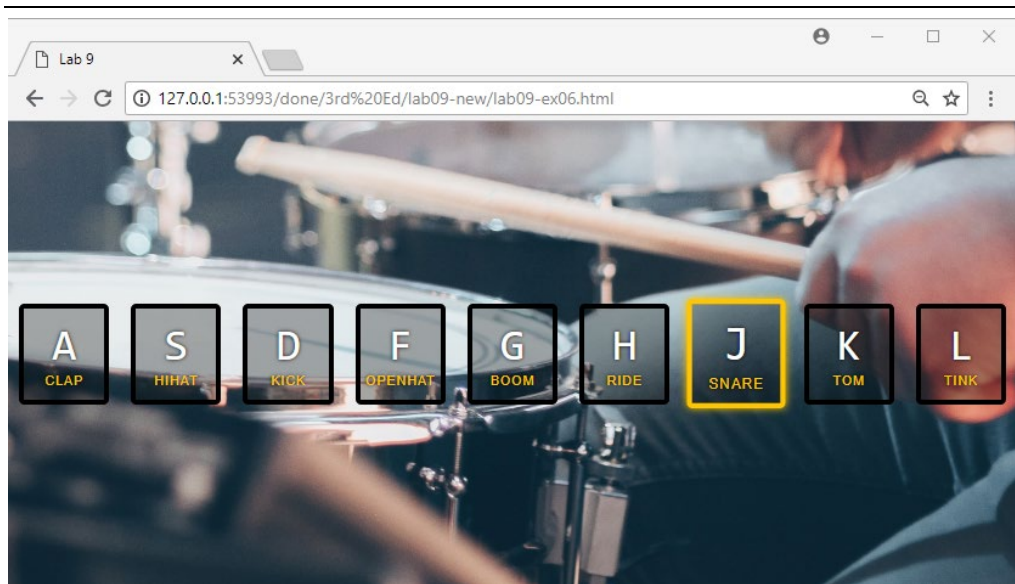


Figure 9.1 – Finished Exercise 9b.1

EXERCISE 9b.2 — DEBUGGING EVENTS

- 1 You are going to continue working with the file from the previous exercise.
- 2 In Chrome, to access the debugger, you have to click on the Sources tab within the Console and then open the JavaScript file (`lab09b-ex01.js`) that you wish to debug.
You will now add a breakpoint to your script by clicking to the left of the first non-comment line within `playSound()` function.
- 3 Try refreshing the page and then pressing the 'g' key. The line with the breakpoint will be highlighted as shown in Figure 9.2. You can now examine the state of local and global variables.

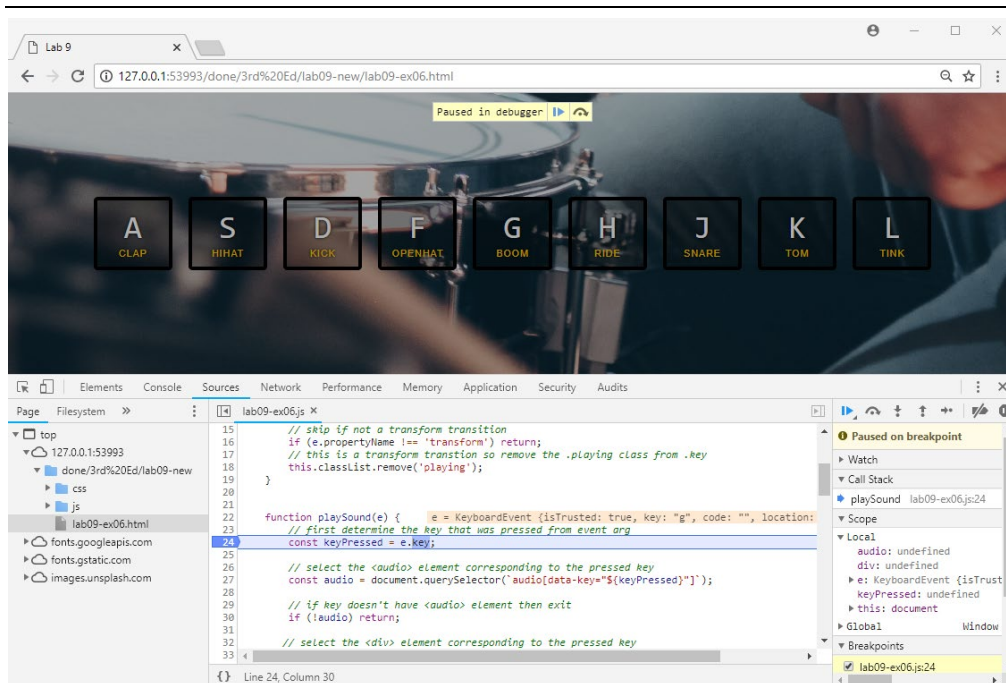


Figure 9.2 – Chrome Debugging

- 4 Once you have set a breakpoint, you can resume execution of the script or continue execution line-by-line using the execution buttons:



- 5 Execute the next line of the code by pressing the Step button  (or pressing F11).
Notice the `keyPressed` variable in the Scope panel now shows a value.
- 6 Add another breakpoint on the last line of the function.
- 7 Press the Resume button  (or press F8).
The intervening lines will be executed and will stop at the new breakpoint. Notice how you can open up and examine the object variables `audio` and `div`.
- 8 Remove breakpoints by clicking on each of the blue breakpoint indicators and refresh the page.

EXERCISE 9b.3 — MEDIA EVENTS

- 1 Examine `lab09b-ex03.html` in your browser. You will be adding some event handlers to turn this into a functional video player.
- 2 Add the following to `lab09b-ex03.js` within the `DOMContentLoaded` handler and test.

```
document.addEventListener("DOMContentLoaded", function() {
  const video = document.querySelector('#vidPlayer');
  const playBtn = document.querySelector('#play');

  playBtn.addEventListener('click', playOrPause);
  video.addEventListener('click', playOrPause);

  function playOrPause() {
    if (video.paused) {
      video.play();
    } else {
      video.pause();
    }
  }
}
```

Notice that you are responding not only to the play/pause button clicks, but also to any click in the video itself.

- 3 To make the play/pause button change depending on its state, add the following and test.

```
document.addEventListener("DOMContentLoaded", function() {
  ...

  video.addEventListener('play', updateButton);
  video.addEventListener('pause', updateButton);

  function updateButton() {
    const icon = video.paused ? symbolPlay : symbolPause;
    playBtn.textContent = icon;
  }
}
```

- 4 To make the volume range slider work, add the following and test.

```
const vol = document.querySelector('#volume');
vol.addEventListener('input', changeVolume);

function changeVolume() {
  video.volume = vol.value;
}
```

Notice that volume is a property of the video element.

- 5 Finally, to make the progress indicator work, add the following and test.

```
const progressBar = document.querySelector('#progressFilled');
video.addEventListener('timeupdate', handleProgress);

function handleProgress() {
  const percent = (video.currentTime / video.duration) * 100;
  progressBar.style.flexBasis = `${percent}%`;
}
```

Notice that the percentage is calculated from two properties of the video element: its duration and its current time position.

EXERCISE 9b.4 — FRAME EVENTS

- 1 Examine `lab09b-ex04.html` in your browser and then examine in text editor.

Notice that after the first six images, the `<img>` elements don't have a `src` attribute but instead have a `data-src` element instead. By doing so, the browser will not request the image files. In this exercise, you will programmatically set the `src` attribute when the `img` elements become visible in the browser.

Instead of using local copies of the images, this example uses versions that are on Cloudinary storage to guarantee some network latency thereby making the fetch visible when testing.

This example based on an online example created by Rahul Nanwani, which is available at <https://imagekit.io/blog/lazy-loading-images-complete-guide/>.

- 2 Add the following to `lab09b-ex04.js`.

```
document.addEventListener("DOMContentLoaded", function() {
  // lazy class indicates images to be loaded eventually
  const images = document.querySelectorAll("img.lazy");

  // do lazy loading for scroll, resize, and window change events
  document.addEventListener("scroll", lazyload);
  window.addEventListener("resize", lazyload);
  window.addEventListener("change", lazyload);

  // flag used to indicate whether we should ignore event
  let lazyloadThrottleTimeout = false;

  function lazyload () {
    // we will add code to this function in the next step
  }
});
```

- 3 Now add the following code to the `lazyload()` function.

```
function lazyload () {
  /* unfortunately, scroll events get triggered too
    frequently, so for performance reasons, we throttle its
    speed so it only runs every 20ms */
  if (lazyloadThrottleTimeout) {
    clearTimeout(lazyloadThrottleTimeout);
  }

  lazyloadThrottleTimeout = setTimeout(function() {
    const scrollTop = window.pageYOffset;
    for (let img of images) {
      // is this image visible?
      if (img.offsetTop < (window.innerHeight + scrollTop)) {
        // yes, then sets its src property, which will make
        // browser request image
        img.src = img.dataset.src;
        img.alt = img.dataset.alt;
        // once it's loaded, no need to load again
        img.classList.remove('lazy');
      }
    }
    // if all images are loaded, no need for this handler anymore
    if (images.length == 0) {
      document.removeEventListener("scroll", lazyload);
      window.removeEventListener("resize", lazyload);
      window.removeEventListener("change",
        lazyload);
    }
  }, 20);
}
```

- 4 Test in browser. Be sure to reduce the width of the browser window so only two or three images are visible in a row.

As you scroll, you should see the image is blank and then becomes downloaded as it becomes visible. After scrolling to the end, scroll up. Notice that the images don't re-download.

COMMON USER INTERFACE TECHNIQUES

Over time, many websites find themselves making use of common user interface techniques. Sometimes, developers will make use of an external library so that they are not re-implementing the wheel. But sometimes, integrating a third-party library introduces a dependency that might cause problems down the road. In the remaining pages, this lab will illustrate some of these common UI designs, not only because they can be useful, but also because they help demonstrate more and more DOM approaches.

The first example illustrates a toast or snackbar. In web design, a toast or snackbar refers to a pop-up message that shows up at the top or bottom of the screen and then disappears. They typically make use of `setTimeout` along with DOM manipulations.

EXERCISE 9b.5 — TOAST / SNACKBAR

- 1 Examine `lab09b-ex05.html` in your browser and then examine in text editor.
You code is going to display a snackbar message after any add to cart button is pressed.

- 2 Add the following to `lab09b-ex05.js` and test by adding a product to the cart.

```
document.addEventListener("DOMContentLoaded", () => {
  const buttons = document.querySelectorAll('button');
  for (let btn of buttons) {
    btn.addEventListener('click', addToCart);
  }

  function addToCart(e) {
    const button = e.target;
    const product = button.dataset.product;

    button.textContent = 'Added!';
    button.style.background =
      'linear-gradient(135deg, #48bb78 0%, #38a169 100%)';

    alert(`Added ${product} to cart`);
  }
});
```

This preparatory code demonstrates the functionality, but uses an old-fashioned alert as if it was still 1997. You will be replacing the alert with a snackbar.

- 3 Add the following markup just before the `</body>` in `lab09b-ex05.html`.

```
<div id="snackbar" class="show">Sample text</div>
```

- 4 Test in browser.

Notice the message box (the snackbar) at the top of the page. You will now programmatically make it appear.

- 5 Edit the just-created markup as follows.

```
<div id="snackbar"></div>
```

- 6 Add the following to `lab09b-ex05.js`.

```
function snackBarHandler(message) {
  const bar = document.querySelector("#snackbar");
  bar.className = "show";
  bar.textContent = message;
  // hide snackbar after four seconds
  setTimeout(() => {
    bar.className = "";
  }, 4000);
}
```

- 7 Remove the `alert` in step 2 with the following and test.

```
snackBarHandler(`${product} added to cart`);
```

The snackbar will appear when a product is added to the cart, and will disappear after four seconds.

Many sites need a way of asking the user for one or more pieces of data. In dedicated desktop applications, this is usually handled by a dialog or modal box/window. Until the introduction of the HTML `<dialog>` element, this type of functionality required a *lot* of JavaScript and CSS, but now, as the next exercise illustrates, this can be done with much less effort.

EXERCISE 9b.6 — CONFIRM DIALOG

- 1 Examine `lab09b-ex06.html` in your text editor.

Notice the dialog element.

- 2 Add the following to `lab09b-ex06.js` and test by clicking the open button.

```
document.addEventListener("DOMContentLoaded", () => {
  const btn = document.querySelector('#opnBtn');
  const dialog = document.querySelector("#myDialog");

  btn.addEventListener('click', () => {
    dialog.showModal();
  });
});
```

Right now, the dialog has no functionality. Let's add some!

- 3 Add the following to `lab09b-ex06.js` and test by clicking the yes and no buttons.

```
dialog.addEventListener('close', (e) => {
  // see the markup to understand where the "yes" comes from
  if (dialog.returnValue == "yes") {
    // do the delete action now user has approved ...
    console.log('deleting now');
  }
});
```

- 4 Add the following to `lab09b-ex06.js` and test by clicking the X button (upper right).

```
const closeBtn = document.querySelector("#closeBtn");
closeBtn.addEventListener('click', () => {
  // make sure we don't do the delete action
  dialog.close("cancel");
});
```

EXERCISE 9b.7 — DIALOG WITH TEXT INPUT

- 1 Examine `lab09b-ex07.html` in your text editor.
Notice the dialog contains an input element.
- 2 Examine `lab09b-ex07.js` in your text editor. It contains the some of the code from the previous exercise.
- 3 Modify `lab09b-ex07.js` as follows and then test the form.

```
// if user clicked confirm then return the input value
const confirmBtn = document.querySelector("#confirmBtn");
const nameInput = document.querySelector("#userName");
confirmBtn.addEventListener("click", (e) => {
  dialog.close(nameInput.value);
});
// if user clicked cancel then return "cancel"
const cancelBtn = document.querySelector("#cancelBtn");
cancelBtn.addEventListener('click', () => {
  dialog.close("cancel");
});
// if user clicked X then return "cancel"
const closeBtn = document.querySelector("#closeBtn");
closeBtn.addEventListener('click', () => {
  dialog.close("cancel");
});
dialog.addEventListener('close', (e) => {
  if (dialog.returnValue != "cancel") {
    // display the user's input
    console.log("form value="+dialog.returnValue);
  }
});
```

Another common use case for the `<dialog>` element is to create modal popups. Perhaps the most common example of a modal popup is for displaying a larger version of an image. The next exercise illustrates such an example.

EXERCISE 9b.8 — MODAL IMAGE POPUPS VIA THE DIALOG

- 1 Examine `lab09b-ex08.html` in the browser and in the text editor.
You will be using the provided `<dialog>` element to provide a modal image popup showing a larger version of the clicked painting.
- 2 Examine `lab09b-ex08.js` in your text editor. It already contains code for dynamically generating the gallery of images from the provided data array using the `<template>` element in `lab09b-ex08.html`.
- 3 Add the following after the provided comment:

```
// TODO: add event handler for each gallery item
item.addEventListener('click', openDialog);
```

This will display the dialog when the user clicks an image.

- 4 Add the following function:

```
function openDialog(e) {
  const index = e.currentTarget.dataset.index;
  const painting = paintings[index];
  dialogImage.src = painting.src;
  dialogImage.alt = painting.title;
  dialogTitle.textContent = painting.title;
  dialogDescription.textContent = painting.description;
  dialogClose.addEventListener('click', closeDialog);

  dialog.showModal();

  // hide scroll bar when displaying dialog
  document.body.style.overflow = 'hidden';
}
```

This populates the dialog with the relevant painting data for the clicked image.

- 5 Add the remaining functions:

```
// Close dialog when clicking on backdrop
dialog.addEventListener('click', (e) => {
  if (e.target === dialog) {
    dialog.close();
  }
});

// show scroll bars again
dialog.addEventListener('close', () => {
  document.body.style.overflow = '';
});
```

- 6 Test. The result should be similar to that shown in Figure 9.3.

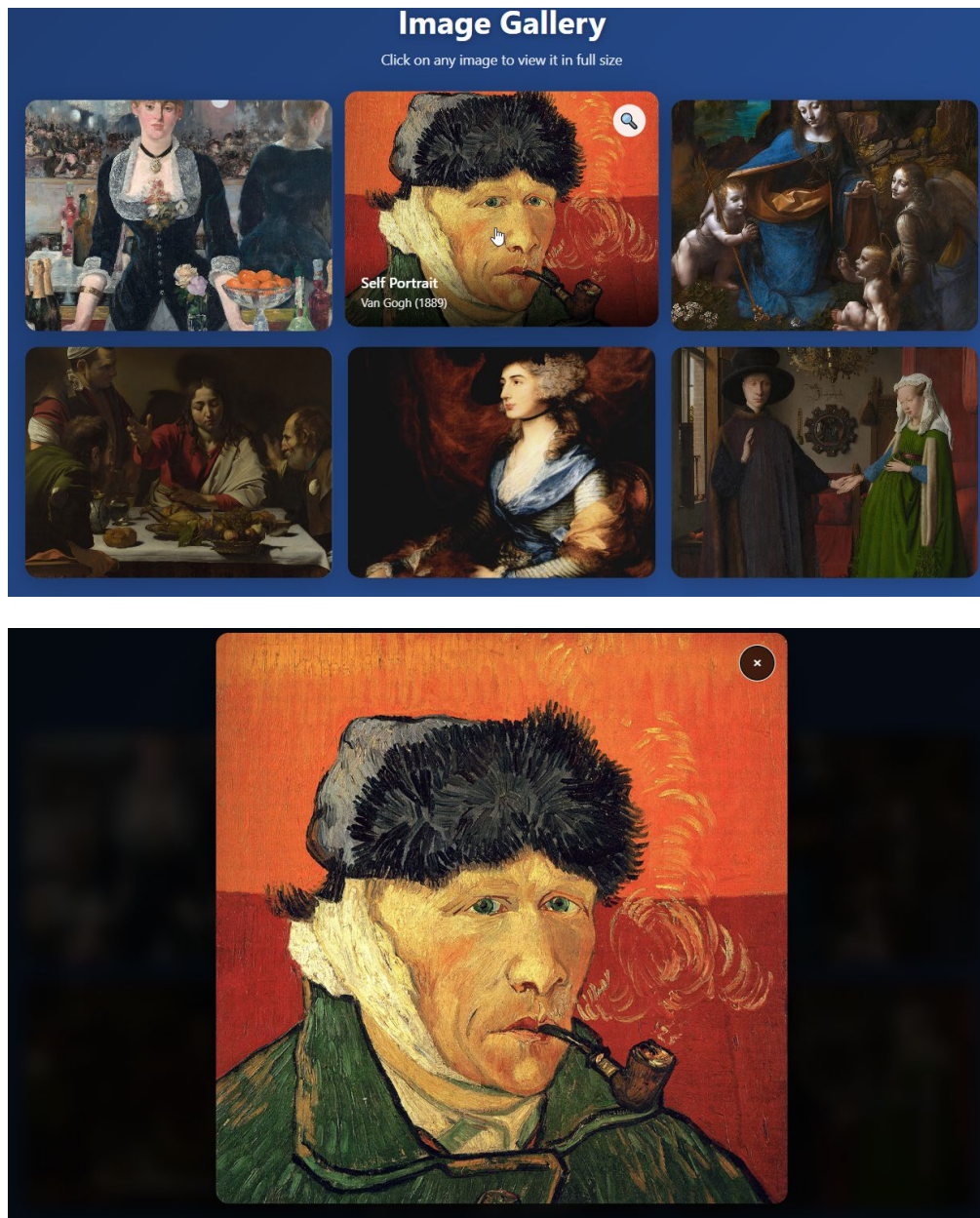


Figure 9.3 – Finished Exercise 9b.8

The next example implements drag and drop functionality, which demonstrates how to work with mouse events. This example is inspired by Lun Dev's excellent article (and replicates much of its styling):

<https://www.lundevweb.com/2024/04/implementing-drag-and-drop.html>

EXERCISE 9b.9 — DRAG AND DROP MOUSE EVENT HANDLING

- 1 Examine `lab09b-ex09.html` in your browser. Try clicking on the + button. Examine in text editor.

You code is going to add the ability to drag these note boxes to different places on the screen.

- 2 Examine `lab09b-ex09.js` in your text editor. It already contains code for dynamically generating a note.
- 3 Add the following code in the indicated spot in the file:

```
// constructor function to represent x,y position of cursor/mouse
function Cursor(x,y) {
    this.x = x;
    this.y = y;
}
// constructor function to represent draggable note
function Note(nodeRef, x, y) {
    this.nodeRef = nodeRef;
    this.x = x;
    this.y = y;
}
// global variables that represent the current cursor and note
let currentCursor = new Cursor(null, null);
let currentNote = new Note(null,null,null);
```

- 4 Add the following event handler:

```
/* we need to handle drag and drop at the document level, not the
object level */

document.addEventListener('mousedown', e => {
    // user can mouse down anywhere .. check if it's in a note
    if (e.target.classList.contains('note')) {
        // change the global cursor and note variables so we
        // manipulate the right one
        currentCursor = new Cursor(e.clientX, e.clientY);
        currentNote = new Note(e.target,
                               e.target.getBoundingClientRect().left,
                               e.target.getBoundingClientRect().top);
    }
})
```

5 Add the following event handlers then test:

```
document.addEventListener('mousemove', e => {
  const noteObj = currentNote.nodeRef;
  if (noteObj == null) return;
  // represent the current position of the mouse
  let movedCursor = new Cursor(e.clientX, e.clientY);
  // calc distance between note's previous position and
  // current mouse/cursor position
  let distance = new Cursor(movedCursor.x - currentCursor.x,
                             movedCursor.y - currentCursor.y);

  // use css to move element
  noteObj.style.left = (currentNote.x + distance.x) + 'px';
  noteObj.style.top = (currentNote.y + distance.y) + 'px';
  noteObj.style.cursor = 'grab';
});

document.addEventListener('mouseup', () => {
  // user can mouse up anywhere .. check if its in a note
  if( currentNote.nodeRef == null) return;
  currentNote.nodeRef.style.cursor = 'auto';
  // the drag is done so indicate that there is no current note
  currentNote.nodeRef = null;
});
```

The result should look similar to that shown in Figure 9.4.

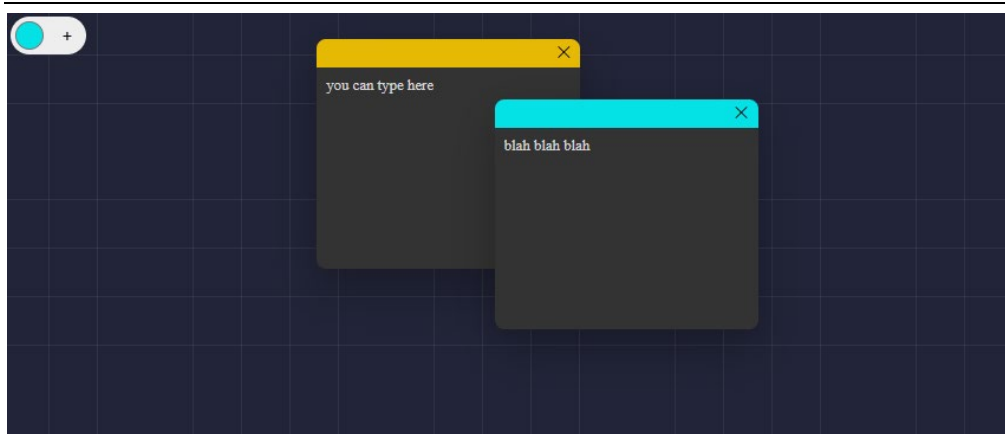


Figure 9.4 – Finished Exercise 9b.9

EXERCISE 9b.10 — IMAGE DRAG AND DROP WITH DISPLAY

- 1 Examine `lab09b-ex10.html` in your browser.

Notice the `<input>` element with `type=file`. The default interface for this element looks like it belongs in a 1998 web site, so this example hides the element and instead implements an interface using other elements but uses the functionality of the `<input>` element.

- 2 Add the following code to `lab09b-ex10.js`.

```
document.addEventListener("DOMContentLoaded", () => {
  const btnChoose = document.querySelector('#btnChoose');
  const fileInput = document.querySelector('#fileInput');
  const uploadArea = document.querySelector('#uploadArea');

  // handle the choose file button
  btnChoose.addEventListener('click', () => {
    // display file picker dialog
    fileInput.click();
  });

  // clicking on the drag area will behave similarly
  uploadArea.addEventListener('click', () => {
    fileInput.click();
  });

  // when user selects a new file, then preview it
  fileInput.addEventListener('change', e => {
    filePreview(fileInput.files[0]);
  });

  // will display a preview of the selected file
  function filePreview(file) {
    if (file) {
      const divPreview =
        document.querySelector('#previewContainer');
      const imgPreview =
        document.querySelector('#imgPreview');

      // this is a built-in object supported by browser JS
      const reader = new FileReader();

      // when the reader has completed reading the file ...
      reader.addEventListener("load", () => {
        // ... then display the image
        divPreview.style.display = 'block';
        imgPreview.src = reader.result;
      });

      // start the file reading process.
      reader.readAsDataURL(file);
    }
  }
});
```

- 3 Test by using the Choose File button and then select an image file on your computer.

It should display the image.

- 4 Add the following code which implements the drag-and-drop functionality.

```
// Drag and drop functionality
uploadArea.addEventListener('dragover', e => {
  e.preventDefault();
  uploadArea.classList.add('dragover');
});

uploadArea.addEventListener('dragleave', e => {
  uploadArea.classList.remove('dragover');
});

uploadArea.addEventListener('drop', e => {
  e.preventDefault();
  uploadArea.classList.remove('dragover');
  filePreview(e.dataTransfer.files[0]);
});
```

- 5 Test by dragging an image file into the drop area. The result should look similar to that shown in Figure 9.5.

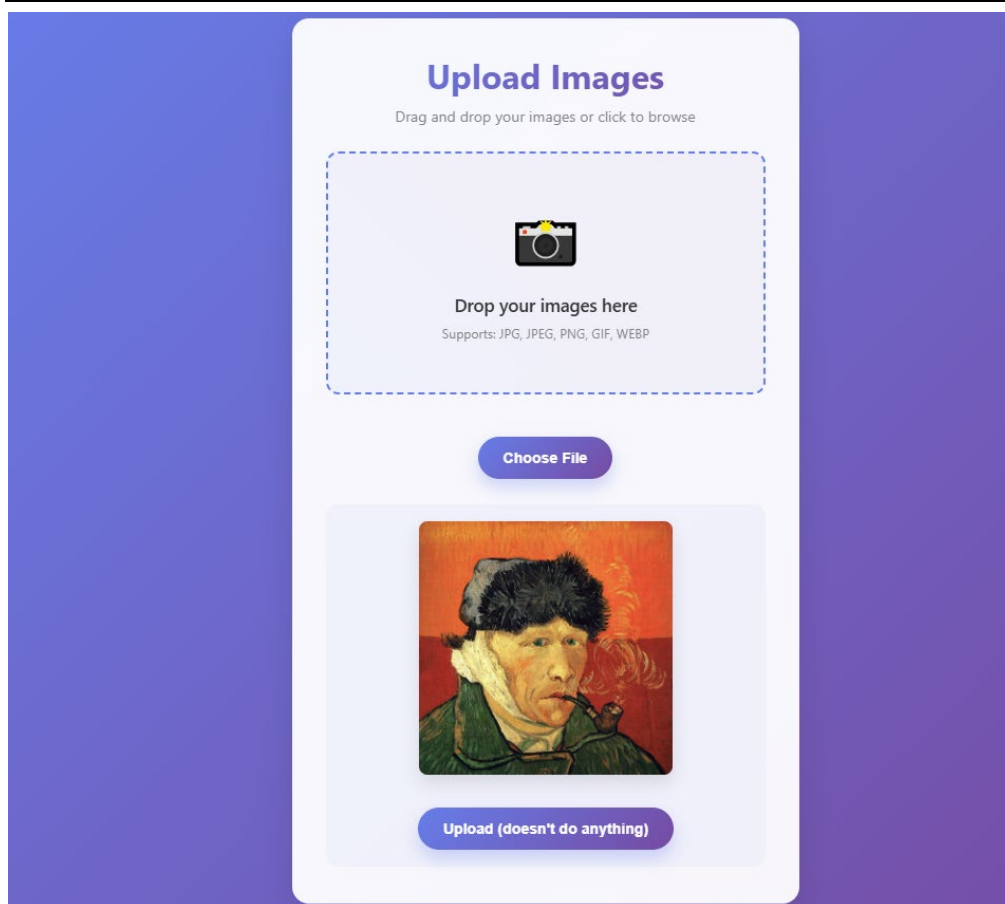


Figure 9.5 – Finished Exercise 9b.10

EXERCISE 9a.11 — MANIPULATING IMAGES USING CSS FILTERS

- 1 Examine `lab09b-ex11.html` in your browser and then in your code editor.
You will be manipulating the shirt image (changing its color and saturation) by programmatically using CSS filters.

- 2 Add the following to `lab09b-ex11.js`:

```
// default will be no hue shift (shirt will remain blue)
let hue = 0;
// default tone will be 75% saturation
let tone = 75;
AdjustShirtImage();

// modify the image using the CSS filter property
function AdjustShirtImage() {
  shirt.style.filter =
    `hue-rotate(${hue}deg) saturate(${tone}%)`;
}
```

Recall from Chapter 7 that if you are setting multiple filters, they have to be included together as a single string separated by spaces.

- 3 Add the following event handlers to `lab09b-ex11.js`:

```
colorOptions.addEventListener('click', e => {
  if (e.target.classList.contains('color-option')) {
    // the hue degree is in the data-hue attribute
    hue = e.target.dataset.hue;
    AdjustShirtImage();
  }
});
toneOptions.addEventListener('click', e => {
  if (e.target.classList.contains('tone-option')) {
    // the saturation % is in the data-tone attribute
    tone = e.target.dataset.tone;
    removeSelectedStyle();
    e.target.classList.toggle('selected');
    AdjustShirtImage();
  }
});
```

Notice it is using event delegation using the same approach as Exercise 9a.11.

- 4 Add the following function and then test by using the color and tone buttons.

```
// remove the selected class from the option buttons
function removeSelectedStyle() {
  const options = document.querySelectorAll('.tone-option');
  for (let opt of options) opt.classList.remove('selected');
}
```

The color and tone buttons should now change the shirt color.

TEST YOUR KNOWLEDGE #1

Examine `lab09b-test01.html`, view in browser, and then open `lab09b-test01.js` in your editor. Modify the JavaScript file to implement the following functionality.

- 1 You are going to need three event handlers. The first will be a click handler for each thumbnail image. In the handler, replace the `src` attribute of the `<img>` element in the `<figure>` so that it displays the clicked thumbnail. Hint: get the `src` attribute of the clicked element and then replace the `small` folder name with `medium` folder name
 - 2 As well, change the `<figcaption>` so that it displays the newly clicked painting's title and artist information. This information is contained within the `alt` and `title` attributes of each thumbnail.
 - 3 Set up an event listener for the input event of each of the range sliders. The code is going to set the `filter` properties on the image in the `<figure>`. Recall from Chapter 7 that if you are setting multiple filters, they have to be included together separated by spaces.
- This listener must use event delegation.** If you don't know what that means, go back and re-do exercises 9a.10 and 9a.11 from the previous lab.
- 4 Add a listener for the click event of the reset button. This will remove the filters from the image by setting the `filter` property to `none`. Also display a snackbar message (see Exercise 9b.5) that gives the user feedback that the filters have been reset.

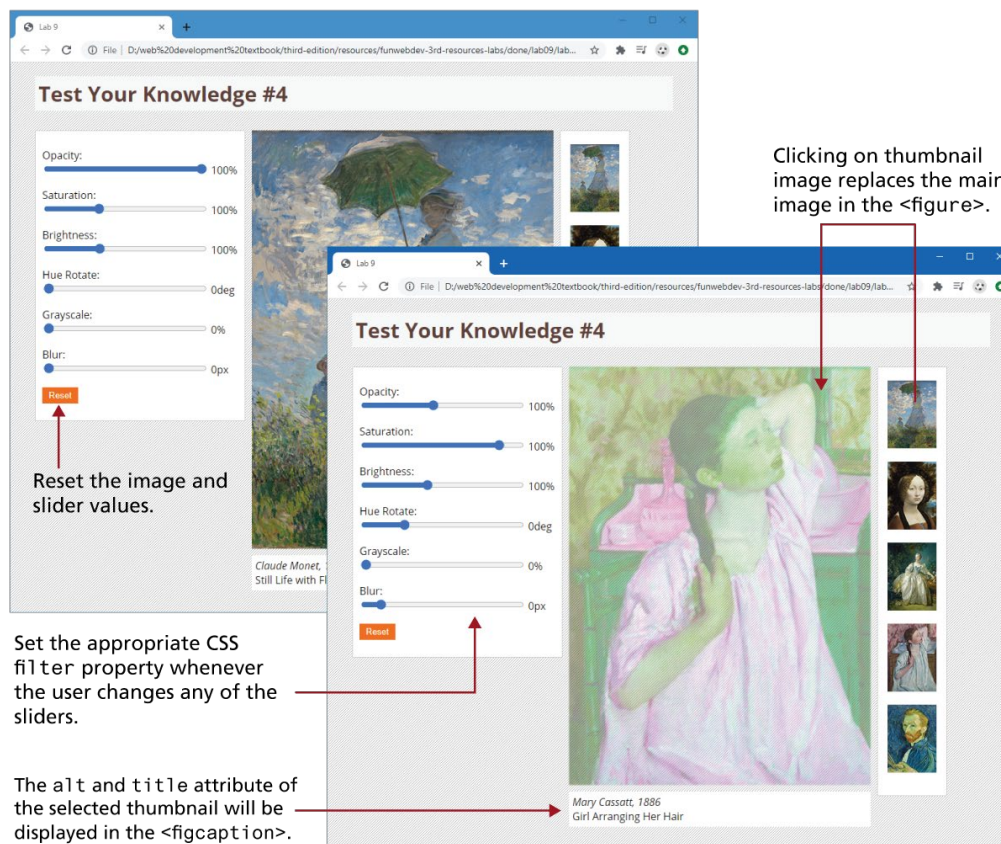


Figure 9.6 –Test Your Knowledge #1